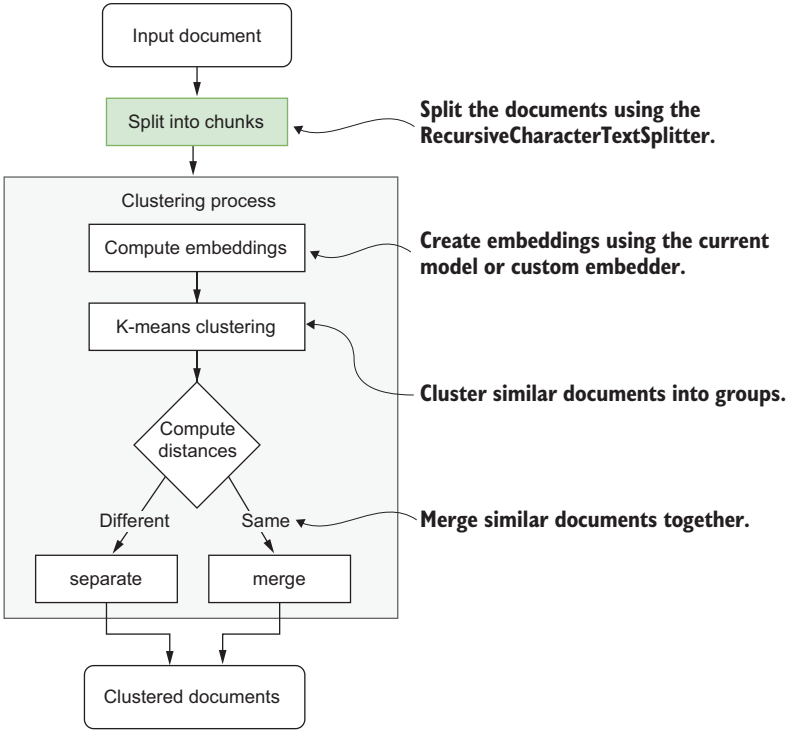


Figure 7.7 The sequential process of document clustering using the k-means algorithm. The input text is first split into chunks using `RecursiveCharacterTextSplitter`, which preserves semantic context. These chunks are then transformed into vector embeddings to capture their semantic meaning. The k-means clustering algorithm groups similar chunks based on their vector representations. A distance-based comparison determines whether chunks should be merged or kept separate, ultimately producing semantically coherent sections.

Listing 7.7 Clustering documents for summarization

```
import { kmeans } from 'ml-kmeans';
import { pipeline } from '@huggingface/transformers';

async function clusterDocuments(documentChunks, similarityThreshold = 0.8) {
  const extractor = await pipeline('feature-extraction',
'Xenova/all-MiniLM-L6-v2');
  const chunkEmbeddings = await Promise.all(
documentChunks.map((chunk) => extractor(chunk)));
  const flattenedEmbeddings = chunkEmbeddings.map(
(embedding) => embedding[0][0].data);

  const numChunks = flattenedEmbeddings.length;
  if (numChunks === 0) {
    throw new Error('No document chunks available for clustering.');
  }
```

Imports the k-means clustering algorithm from ml-kmeans library for document clustering

Initializes feature extraction pipeline using the MiniLM model for generating document embeddings